Noncontiguous Memory and Translation

READ, RECALL, THEN PRACTISE

StudyNotes original PDF pages **39-47**. Notes begin on page 2 of this pack. Original content and annotations are preserved.

- Translate segment/offset and virtual page/offset addresses.

- Calculate address fields and page-table size with correct units.

- Compare page tables and TLB; distinguish a TLB miss from a page fault.

SELECTED PRACTICE

WR: address bits and translation / original PDF pages 7, 12, 13, 14

Each practice section has a source page and question guide. Mixed-topic source pages may appear in more than one pack.

Lecture 5 — Noncontiguous Memory Allocation

อ้างอิง: Lecture 5 หน้า 1–30 | Segmentation, Paging, page table, TLB และ hybrid translation

5.1 ทำไมต้อง Noncontiguous Allocation

Contiguous allocation ใน Lecture 4 กำหนดให้ address space ของ process อยู่ติดกันทั้งก้อนใน physical memory วิธีนี้ทำให้เกิด fragmentation และต้องกันพื้นที่ให้ช่องว่างใน address space เช่น บริเวณระหว่าง heap กับ stack ด้วย Lecture 5 เสนอวิธีจัดสรรหลักสองแบบ:

- **Segmentation**: แบ่ง logical address space เป็นส่วนที่มีความหมายและขนาดไม่เท่ากัน เช่น code, heap, stack
- **Paging**: แบ่ง logical address space เป็น pages ขนาดเท่ากัน และ physical memory เป็น frames ขนาดเท่ากับ page

5.2 Segmentation Scheme

> **เนื้อหาจาก Lecture**
>
> แบ่ง logical address space ออกเป็น **segments** ที่มีขนาดไม่เท่ากัน แต่ละ segment ยังเป็นช่วงต่อเนื่องของ address space จากนั้นขยาย Boundary Scheme ให้มี **Base + Bounds** แยกสำหรับแต่ละ segment ตัวอย่างที่แบ่งเป็น code, heap และ stack จึงต้องมี Base/Bounds สามคู่ใน MMU

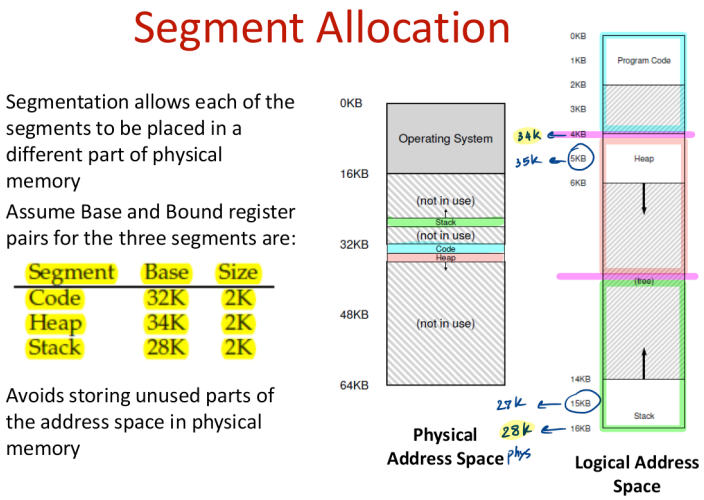
5.2.1 Segment Allocation

ข้อดีคือ code, heap และ stack วางคนละตำแหน่งใน physical memory ได้ จึงไม่ต้องสำรองพื้นที่ของช่องว่างระหว่าง heap กับ stack ไว้ทั้งหมด

ตัวอย่างใน Lecture ใช้ค่าดังนี้:

Segment	Base	Size/Bounds
Code	32 KB	2 KB
Heap	34 KB	2 KB
Stack	28 KB	2 KB

ภาพประกอบ 5.1 Lecture 5 หน้า 5 — Segment Allocation



จุดสังเกต

- code/heap/stack วางคนละตำแหน่งใน physical memory ได้
- แต่ละ segment มี Base + Size/Bounds ของตัวเอง
- benefit หลักคือไม่ต้องเก็บ unused gap ระหว่าง heap กับ stack ทั้งก้อนใน RAM

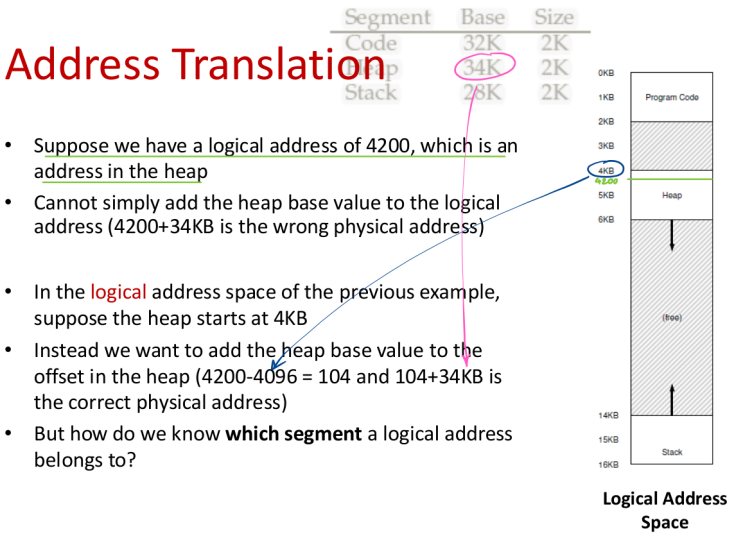
ที่มา: Lecture 5 หน้า 5 พร้อมหมายเหตุเรื่อง base/offset จากสไลด์

5.2.2 Segmentation Address Translation

Logical address ต้องระบุว่าอยู่ segment ใดและ offset ภายใน segment เท่าใด ไม่สามารถเอา logical address ทั้งก้อนไปบวก Base ของ segment ตรง ๆ

> **ตัวอย่างคำนวณ**
>
> ตัวอย่างใน Lecture ให้ logical address เท่ากับ 4200 ซึ่งอยู่ใน heap โดย heap เริ่มที่ logical address 4KB=4096 และมี physical Base เท่ากับ 34KB
> 1. หา offset ภายใน heap:
>
> $$offset = 4200 - 4096 = 104 \text{ B}$$
>
> 2. ตรวจ bounds: $104 < 2$KB ดังนั้น address นี้อยู่ในขอบเขตที่อนุญาต 3. แปลงเป็น physical:
>
> $$PA = 34\text{KB} + 104 = 34816 + 104 = 34920 \text{ B}$$
>
> อย่านำ $4200 + 34$KB มาบวกกันโดยตรง เพราะ 4200 เป็น logical address ของทั้ง process ไม่ใช่ offset ภายใน heap

ภาพประกอบ 5.2 Lecture 5 หน้า 6 — Segmentation Address Translation Example



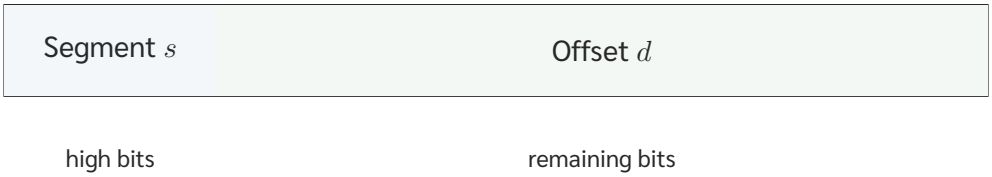
จุดสังเกต

- logical address 4200 อยู่ใน heap ซึ่งเริ่มที่ 4KB จึงได้ offset เท่ากับ $4200 - 4096 = 104$
- อย่าบวก $4200 + Base_{heap}$ โดยตรง เพราะ 4200 ยังเป็น address ของทั้ง logical space
- คำนวณ physical address จาก $Base_{heap} + 104$

ที่มา: Lecture 5 หน้า 6 คงลูกศรและการเน้นในสไลด์ไว้เพื่อชี้ตำแหน่งที่ใช้คำนวณ offset

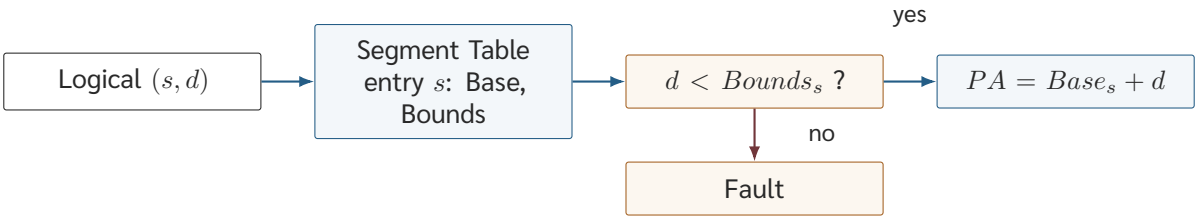
5.2.3 Segment Number + Offset

Lecture เสนอให้ high bits ของ logical address encode segment number และ remaining bits เป็น offset. ถ้ามีสาม segments ใช้สอง bits แยก segment ได้ Stack มีทิศทาง grow backward จึงต้องมีข้อมูลเพิ่มเพื่อบอก growth direction ตาม slide model



5.2.4 Segment Table

เมื่อมีหลาย segments, Base/Bounds ถูกจัดเป็น **Segment Table**; แต่ละ entry เป็น Base+Bounds pair และแต่ละ process มี segment table ของตัวเอง Logical address (s, d) ใช้ s เลือก entry แล้วตรวจ d กับ Bounds ก่อนคำนวณ $Base_s + d$.



หมายเหตุในสไลด์หน้า 8 ระบุว่า Segment Table อยู่ใน RAM การค้นตารางจึงเพิ่มการเข้าถึง memory อีกครั้ง ประเด็นเรื่อง access overhead นี้จะเห็นชัดขึ้นเมื่อเรียน Paging

5.2.5 Protection, growth direction, fine/coarse grain

Lecture ระบุ fields/issue เพิ่ม:

- growth direction ของ segment (สำคัญกับ stack)
- permission เช่น read/write/execute เพื่อ support sharing/protection
- coarse-grained segmentation = segments น้อยและใหญ่; fine-grained = segments มากและเล็ก

Fine-grained segmentation ยังมี external fragmentation เพราะ segment sizes ไม่เท่ากัน

5.2.6 การจัดสรร Memory ให้ Segments

OS เก็บ **free list** ของ holes แล้วเลือกพื้นที่ด้วยวิธีต่อไปนี้:

First Fit
เลือก hole แรกที่มีขนาดพอ เหมาะกับการค้นหาให้เร็ว

Best Fit
เลือก hole ที่เหลือพื้นที่ว่างน้อยที่สุด เพื่อลดพื้นที่เหลือ แต่ก็อาจสร้าง holes ขนาดเล็กหลายจุด

Worst Fit
เลือก hole ที่เหลือพื้นที่ว่างมากที่สุด หวังว่าจะไม่เหลือ holes เล็ก ๆ จำนวนมากเหมือน Best Fit

เพราะ external fragmentation ยังเกิดได้ จึงอาจต้อง coalescing/compaction เช่น Lecture 4

5.3 Paging

> **เนื้อหาจาก Lecture**
>
> Paging แบ่ง logical address space เป็น **pages** ขนาดเท่ากัน และแบ่ง physical memory เป็น **frames** ที่มีขนาดเท่ากับ page ดังนั้นหนึ่ง frame จึงบรรจุได้หนึ่ง page Lecture ระบุขนาด page ที่พบได้ทั่วไปไว้ที่ 512–16K bytes แต่ละ process มี **page table** สำหรับจับคู่ page กับ frame

ข้อดีของ Paging คือจัดการ free space ได้ง่าย เพราะ frames มีขนาดเท่ากัน และไม่เกิด external fragmentation แบบ segmentation อย่างไรก็ตาม page สุดท้ายอาจเหลือพื้นที่ว่างภายใน frame ทำให้เกิด internal fragmentation ได้

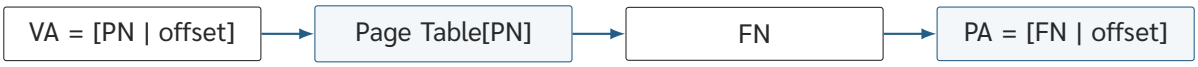
> **จุดที่มักสับสน**
>
> **Page ต่างจาก Frame** Page เป็นหน่วยใน logical/virtual address space ส่วน Frame เป็นช่องขนาดเท่ากันใน physical memory Page Table ใช้บอกว่าแต่ละ page อยู่ใน frame ใด

5.4 Page Table และการแปล Address

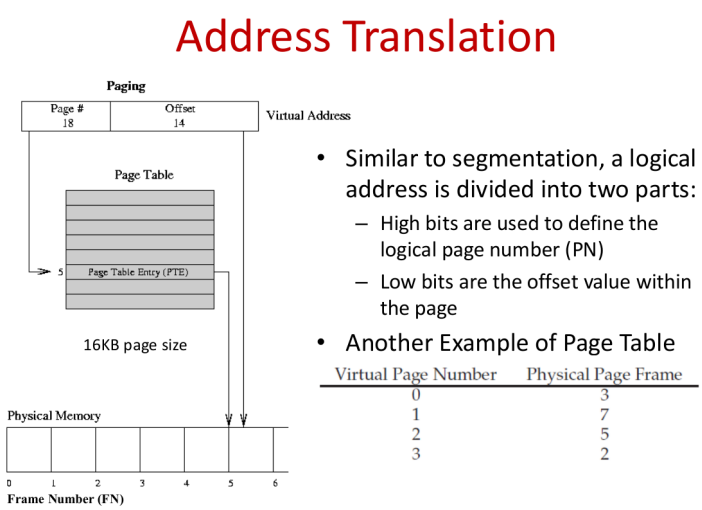
แบ่ง logical address ออกเป็นส่วนต่าง ๆ ดังนี้:

- high bits แทน **Page Number (PN/VPN)**
- low bits แทน **Page Offset**

ใช้ PN ค้นหา Frame Number (FN/PFN) ใน Page Table แล้วประกอบ physical address จาก frame number และ offset เดิม ค่า offset ไม่เปลี่ยน เพราะ page และ frame มีขนาดเท่ากัน



ภาพประกอบ 5.3 Lecture 5 หน้า 15 — Paging Address Translation



จุดสังเกต

- แยก logical address เป็น **Page Number** + **Offset**
- ใช้ Page Number เป็น index ค้น PFN ใน Page Table ส่วน offset คงเดิม
- ประกอบ Physical Address จาก PFN และ offset เดิม

ที่มา: Lecture 5 หน้า 15.

5.4.1 คำนวณจำนวน Bits ของ Paging

Lecture หน้า 16 ใช้ logical address ขนาด 32 bits โดย 20 bits ด้านสูงเป็น page number และ 12 bits ด้านต่ำเป็น offset ดังนั้น page size คือ $2^{12} = 4096$ B หรือ 4KB

ตัวอย่างคำนวณ

ตัวอย่างฝึกทำเพิ่มเติม ตัวอย่างนี้สร้างขึ้นเพื่อฝึกวิธีคิด ไม่ใช่ค่าจาก Lecture กำหนด VA ขนาด 32 bits, page size 4KB และ $VA = 0\text{x}12345$ โดยสมมติว่า Page Table จับคู่ $PN = 0\text{x}12$ กับ frame 7

1. จำนวน offset bits คือ $\log_2 4096 = 12$ bits
2. คำนวณ $PN = \lfloor VA/4096 \rfloor = 0x12 = 18$
3. คำนวณ $offset = VA \bmod 4096 = 0x345 = 837$
4. ค้น Page Table แล้วได้ PFN=7
5. ประกอบเป็น $PA = 7 \times 4096 + 837 = 29509 = 0\text{x}7345$

ให้ทำตามลำดับนี้: **แยก address field ค้น mapping คง offset เดิม แล้วประกอบ physical address**

5.5 Page Table Size Problem

Lecture หน้า 16 สมมติว่าแต่ละ PTE ใช้ 32 bits เก็บ frame number และอีก 1 valid bit รวมเป็น 33 bits ต่อ entry เมื่อมี 2^{20} PTEs:

$$2^{20} \times 33 = 34{,}603{,}008 \text{ bits}$$

แปลงเป็น byte โดยหาร 8:

$$\frac{34{,}603{,}008}{8} = 4{,}325{,}376 \text{ bytes} \approx 4.125 \text{ MiB} \approx 4.33 \text{ MB}$$

สไลด์ย้ำว่าขนาดนี้เป็นของ process เดียว จึงต้องหาโครงสร้างที่ลดพื้นที่ของ page table โดยเก็บ page table ไว้ใน kernel memory และใช้ Page Table Base Register กับ Page Table Size Register ระบุตำแหน่งและขนาด

ภาพประกอบ 5.4 Lecture 5 หน้า 16 — How Large is the Page Table?

How Large is the Page Table?

- Consider a 32 bit logical address where the 20 MSBs define page number and the 12 LSBs define page offset (4KB page)
- Assume 32 bits to keep all the FNs (frame number) in physical memory
- Use 1 bit for a valid bit (space used)
- So, 2^{20} is the number of PTEs and the size of the page table is 2^{20} * 33 (~34 MByte)
- This is just one process!
- As the tables are so large, they are held in kernel memory – a Page Table Base Register contains the physical address of the start of the page table and a Page Table Size Register its size

	V	Physical Page
0		32 bits
1		
2		
	⋮	⋮
$2^{20}-1$		

100 process → 3GB
1000 process → 34 GB

16

จุดสังเกต

- เมื่อ VA มี 32 bits และ page size 4KB จะใช้ 12 bits เป็น offset และ 20 bits เป็น page number จึงมี 2^{20} PTEs ต่อ process
- $2^{20} \times 33$ ได้หน่วยเป็น bit; เมื่อหาร 8 จะได้ประมาณ 4.125 MiB ต่อ process
- ประเด็นของตัวอย่างคือ **linear page table ใช้พื้นที่มาก** เมื่อ address space มีขนาดใหญ่

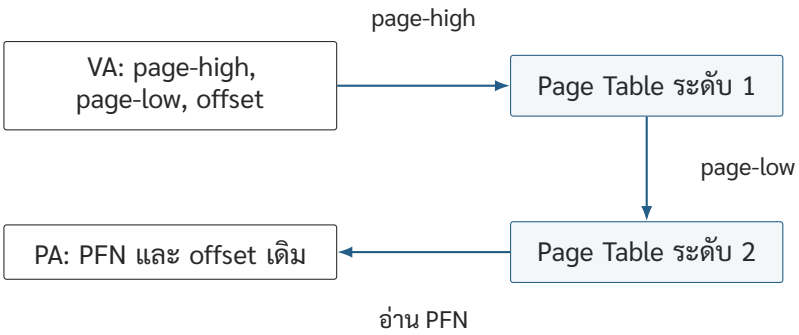
ที่มา: Lecture 5 หน้า 16; หนังสือคำนวณหน่วย bit-to-byte เพิ่มจากนิพจน์บนสไลด์

จุดที่มักสับสน

ตรวจหน่วยกับตัวเลขที่สไลด์พิมพ์ สไลด์ระบุ ~ 34 MByte แต่ค่าที่คูณได้คือ 34,603,008 **bits**; เมื่อหาร 8 จะได้ 4,325,376 bytes หรือประมาณ 4.125 MiB. ตัวเลข 34 MByte บนสไลด์จึงไม่สอดคล้องกับหน่วยของนิพจน์ที่พิมพ์ไว้

5.6 Multi-Level Page Table

Two-Level Paging แบ่ง page table ออกเป็นหลายระดับ โดยใช้ส่วน high ของ Page Number ค้น first-level table และส่วน low ค้น second-level table ไม่จำเป็นต้องสร้าง second-level table ทุกส่วนไว้พร้อมกัน จึงประหยัด memory ในบริเวณของ address space ที่ยังไม่ได้ใช้ โดยทั่วไปออกแบบให้ second-level page table หนึ่งชุดมีขนาดพอดีกับหนึ่ง page



ข้อแลกเปลี่ยนคือใช้ memory เก็บตารางน้อยลง แต่การแปล address อาจต้องอ่าน memory หลายครั้งหากไม่พบข้อมูลใน TLB

5.7 Inverted Page Table

Lecture ตั้งข้อสังเกตว่า frames มักมีจำนวนน้อยกว่า virtual pages **Inverted Page Table** จึงมีหนึ่ง entry ต่อ physical frame โดยเก็บ logical page number และ PID ของ process เจ้าของ page เมื่อต้องการแปล address ระบบค้น entry ที่ตรงกับ (PID, logical page) แล้วใช้ index ของ entry นั้นเป็น physical frame number

วิธีนี้ช่วยลดขนาด table แต่ใช้เวลาแปล address มากขึ้นเพราะต้องค้นหา จึงเป็นตัวอย่างของ time-space trade-off

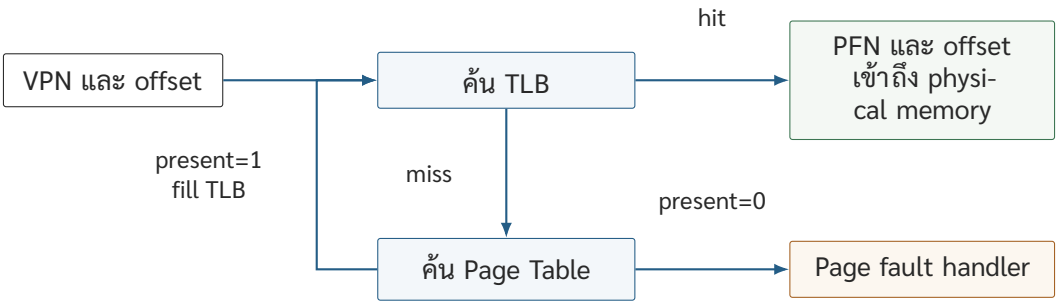
5.8 Hashed Page Table

ระบบ hash logical page number ไปยัง bucket หนึ่ง แต่ละ bucket เชื่อมรายการ entry ซึ่งเก็บ logical page number, mapped frame และ pointer ไปยัง entry ถัดไป ระบบจึงค้นเฉพาะ chain ที่เกี่ยวข้อง และไม่ต้องมี entry สำหรับทุก logical page Lecture ระบุว่าวิธีนี้พบได้ใน address space ที่ใหญ่กว่า 32 bits

5.9 Translation Lookaside Buffer (TLB)

เนื่องจาก Page Table อยู่ใน main memory การใช้ linear table จึงเพิ่ม memory lookup ให้กับ logical address ทุกครั้ง แม้ multi-level table จะลดขนาดตาราง แต่ก็อาจเพิ่มจำนวน lookup Lecture จึงแนะนำ **Translation Lookaside Buffer (TLB)** ซึ่งเป็น hardware cache ภายใน MMU

> **เนื้อหาจาก Lecture**
>
> TLB เป็น hardware cache ที่เก็บ translation ระหว่าง logical page กับ physical frame แบบ **fully associative** และอาศัย locality of reference Hardware จะค้น TLB ก่อนเข้า Page Table:
> - หาก TLB hit ระบบจะได้ translation ทันที
> - หาก TLB miss ระบบจะไปค้น Page Table แล้วนำ translation ที่พบมาเก็บใน TLB
>
> ตามแบบจำลองใน Lecture TLB เก็บ translation ของ process ปัจจุบัน เมื่อเกิด context switch ระบบจึงล้าง entries หรือทำเครื่องหมายว่าใช้ไม่ได้



เมื่อ TLB เต็ม Lecture ระบุว่าสามารถแทน entry ที่ไม่ได้ใช้นานที่สุดด้วย Least-Recently-Used (LRU) ได้

> **เชื่อมกับ Computer Architecture**
>
> TLB ใช้แนวคิด cache/locality และ associative lookup จาก Computer Architecture แต่ **TLB ไม่ใช่ data cache**: มัน cache address translations. TLB hit/miss บอกว่ามี translation หรือไม่ ไม่ได้บอกว่า page อยู่ RAM หรือ disk — ประเด็น present/page fault จะมาใน Lecture 6.

> **จุดที่มักสับสน**
>
> **TLB miss ต่างจาก Page Fault.** TLB miss สามารถไป Page Table แล้วพบ valid/present mapping ใน memory ได้ตามปกติ Page Fault เป็นคนละ condition ที่ Virtual Memory ตรวจว่า page ที่ต้องใช้ไม่ได้ resident ตาม state ของ PTE.

ภาพประกอบ 5.5 Lecture 5 หน้า 22 — Translation Lookaside Buffer (TLB)

Translation Lookaside Buffer

- The TLB is simply a hardware cache of popular logical page number to physical page frame translations
  - Fully associative – hardware will search entire TLB in parallel
- Based on idea of locality of reference
- The hardware looks in TLB first
  - If it has the required page, the translation is very quick
  - Otherwise, the page table is accessed in the normal way and the translation stored in the TLB
- The TLB contains address translations for the current process
  - The TLB is flushed (entries marked invalid) on a context switch

22

จุดสังเกต

- TLB cache **address translations** ไม่ใช่ cache ของ data bytes
- TLB hit ลดการไปอ่าน Page Table ใน main memory
- TLB miss ยังอาจมี page resident อยู่ใน RAM; ดังนั้น **TLB miss ต่างจาก Page Fault**

ที่มา: Lecture 5 หน้า 22.

5.10 Page Size Trade-offs

ตาม Lecture:

	Page ขนาดเล็ก	Page ขนาดใหญ่
Fragmentation	ลด internal fragmentation	internal fragmentation อาจมากขึ้น
Working set / memory	อาจใช้ memory เก็บ working set น้อยลง เหลือให้ process อื่นมากขึ้น	ต้องโหลดและจัดการข้อมูลเป็นก้อนใหญ่ขึ้น
Page table	มี pages มากขึ้น จึงต้องมี entries มากขึ้น	จำนวน entries และ table overhead ลดลง
TLB reach	แต่ละ entry ครอบคลุมพื้นที่เล็ก	แต่ละ TLB entry ครอบคลุมพื้นที่มากขึ้น
I/O	อาจต้องทำ I/O หลายครั้ง	อาจลดจำนวน I/O ที่ใช้โหลด working set

ภาพประกอบ 5.6 Lecture 5 หน้า 25 — Page Size Trade-off

Page Size

- Small Page Size
  - Reduces internal fragmentation
  - Can reduce the amount of memory required to contain a process's working set
  - More memory is available to other processes
- Large Page Size
  - Reduces wasted memory from table fragmentation
  - Enables each TLB entry to map a larger region of memory, improving performance
  - Reduces the number of I/O operations the system performs to load a process's working set into memory

25

จุดสังเกต

- page ขนาดเล็กช่วยลด internal fragmentation และจัด working set ได้ละเอียดขึ้น แต่ทำให้ Page Table ใหญ่ขึ้น
- page ขนาดใหญ่ทำให้ TLB entry ครอบคลุม memory ได้มากขึ้น และอาจลดจำนวน I/O ที่ต้องใช้โหลด working set
- ไม่มี page size ที่ชนะทุก metric; นี่คือ design trade-off ไม่ใช่คำตอบ ยิ่งใหญ่ยิ่งดี

ที่มา: Lecture 5 หน้า 25 หมายเหตุเปรียบเทียบ internal fragmentation กับขนาด page table

5.11 เปรียบเทียบ Segmentation กับ Paging

ประเด็น	Segmentation	Paging
ขนาดหน่วย	segment มีขนาดไม่เท่ากัน	page/frame มีขนาดเท่ากัน
ความหมาย	แบ่งตามส่วนของงาน เช่น code/heap/stack	แบ่งเป็นช่วงขนาดคงที่
External fragmentation	เกิดได้	ไม่เกิดตามแบบจำลองใน Lecture
Internal fragmentation	ไม่เกิดตามการเปรียบเทียบใน Lecture	เกิดได้ โดยเฉพาะเมื่อ page ใหญ่
Metadata	ใช้ segment table ขนาดเล็กกว่า	page table อาจมีขนาดใหญ่
การจัดการ	จัดสรรหน่วยต่างขนาดได้ยากกว่า	จัดการ free frame ได้ง่ายและยืดหยุ่นกว่า

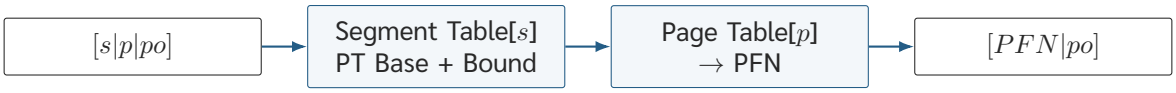
5.12 Hybrid Segmentation + Paging

Hybrid scheme แบ่ง address space เป็น logical segments แล้วแบ่งแต่ละ segment เป็น pages อีกชั้นหนึ่ง แทนที่จะมี page table ใหญ่เพียงชุดเดียว ระบบมี page table แยกสำหรับแต่ละ segment โดย Base เก็บ physical address ของ page table และ Bounds ระบุขนาดของตารางนั้น

Logical address แบ่งสามส่วน:

$$[\ Segment\ s\ |\ Page\ p\ |\ Page\ Offset\ po\ ]$$

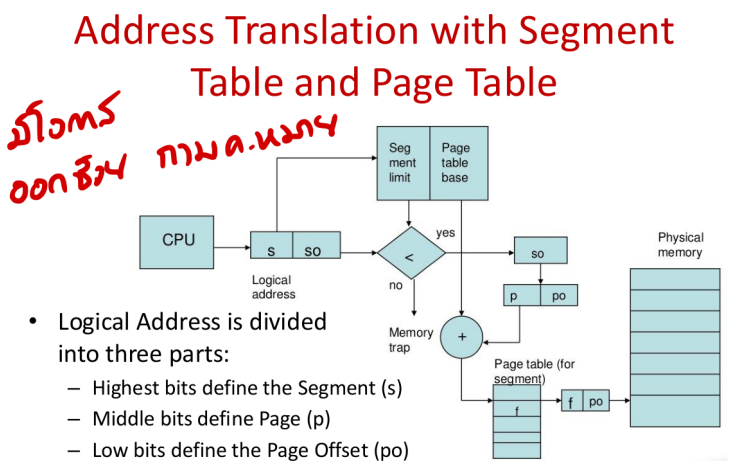
แต่ละ process มี segment table หนึ่งชุด และมี page tables หลายชุดตามจำนวน segments



ข้อดีของวิธีนี้คือใช้ memory เก็บตารางน้อยกว่า linear page table เพราะช่องว่างที่ยังไม่จัดสรร เช่น ระหว่าง heap กับ stack ไม่ต้องมี page-table entry และยังใช้ TLB ได้ ข้อจำกัดคือ page tables ของแต่ละ segment มีขนาดไม่เท่ากัน จึงยังเกิด external fragmentation ได้ นอกจากนี้ Segmentation ยังตั้งอยู่บนสมมติฐานบางอย่างเกี่ยวกับรูปแบบการใช้ address space

> **อ่านภาพและหมายเหตุ**
>
> ข้อความเขียนมือสีแดงบางส่วนในสไลด์หน้า 27 และ 29 อ่านได้ไม่ชัด จึงไม่ถอดความส่วนที่ไม่แน่ใจ เนื้อหาในบทนี้อ้างเฉพาะข้อความที่ตรวจสอบได้จากต้นฉบับ

ภาพประกอบ 5.7 Lecture 5 หน้า 29 — Hybrid Segmentation + Paging Translation



จุดสังเกต

- แบ่ง address เป็นสามส่วน: segment, page ภายใน segment และ page offset
- Segment Table ชี้ไปยัง Page Table ของ segment นั้น แล้ว Page Table แปลง page เป็น frame
- วิธีนี้รวมการแบ่งตามความหมายของ Segmentation กับ page ขนาดคงที่ แต่มี lookup และ metadata ซับซ้อนขึ้น

ที่มา: Lecture 5 หน้า 29 ข้อความเขียนมือสีแดงบางส่วนอ่านไม่ชัด จึงถอดเฉพาะส่วนที่ตรวจสอบได้

5.13 Chapter Summary

สรุปท้ายบท

- Segmentation แบ่ง logical regions ขนาดไม่เท่ากันและใช้ Base/Bounds ต่อ segment จึงไม่ต้องจัดสรรช่องว่างใน address space แต่ยังเกิด external fragmentation
- Paging แบ่ง pages และ frames ให้มีขนาดเท่ากัน แต่ละ process มี Page Table สำหรับจับคู่ page กับ frame จัดการ free space ได้ง่ายและไม่เกิด external fragmentation แต่อาจเกิด internal fragmentation และมี page-table overhead
- Address translation ของ paging คง offset เดิมและแปลง Page Number เป็น Frame Number
- Multi-level, Inverted และ Hashed Page Table ลดพื้นที่ metadata ด้วยข้อแลกเปลี่ยนต่างกัน
- TLB เป็น fully-associative hardware cache สำหรับเก็บ translations และอาศัย locality ในแบบจำลองของ Lecture จะ flush เมื่อ context switch
- การเลือก page size มีผลต่อ fragmentation, page-table size, TLB reach และ I/O overhead
- Hybrid scheme ใช้ segment table ร่วมกับ page table ของแต่ละ segment เพื่อประหยัดพื้นที่ตาราง แต่ยังมีข้อจำกัดของ Segmentation

กลไกสำคัญ

MMU translation, การค้น segment/page table, Base/Bounds check, TLB associative lookup และผลของ context switch

นโยบายสำคัญ

First/Best/Worst Fit สำหรับ segments, TLB replacement (แสดง LRU ใน Lecture) และการเลือก page size

Formula / Calculation Recap

Segmentation: $PA = Base_s + offset$ after bounds check.
Paging: $PN = \lfloor VA/PageSize \rfloor$, $offset = VA$ mod $PageSize$, $PA = PFN \times PageSize + offset$.

ComArch Connections

MMU, physical address, associative lookup, locality, hardware cache concept, register/state changes on context switch.

Common Confusions

- Segment ต่างจาก Page; unequal/meaningful vs fixed-size
- Page ต่างจาก Frame
- Page Table ต่างจาก TLB
- TLB miss ต่างจาก Page Fault
- Paging removes external fragmentation but can create internal fragmentation
- Inverted Page Table มี entry ต่อ physical frame ไม่ใช่ต่อ virtual page

Quick Revision Checklist

☐ ทำ segment+offset translation และ bounds check ได้
☐ ทำ page-number/offset bit split และ physical reconstruction ได้
☐ อธิบาย linear vs 2-level vs inverted vs hashed page table ได้
☐ trace TLB hit และ TLB miss path ได้
☐ เปรียบเทียบ Segmentation vs Paging แบบ fragmentation/metadata/management ได้
☐ trace Hybrid $[s|p|po]$ ผ่าน segment table และ page table ได้

Practice: WR: address bits and translation

ORIGINAL SOURCE

WR / Written question collection (17 pages)

Full source filename is listed under this code in PRACTICE_INDEX.md.

Original PDF page(s): **7, 12, 13, 14**

Original p7: 140 pages. p12: page tables. p13: 64/32-bit fields. p14: segmentation; state ID and stack-direction assumptions before calculating.

ANSWER ANNOTATIONS

Handwritten, highlighted and pre-selected answers are part of the source and are not certified solutions. Hide them while practising. Recomputed examples and explicit assumptions are in CHECKED_DRILLS and the main study guide.

(You have 10 minutes)

3. (4 points) Consider a paging system with a virtual address space of 140 pages of 1024 Bytes each, which is mapped onto a physical memory of 64 frames.

   a. How many bits are there in the virtual address?

   b. How many bits are there in the physical address?

LBSz $2^{10} \times 140 = 143360$

MBS = $2^x = 143360$

$\log 2^x = \log 143360$

$x = \frac{\log 143360}{\log 2}$

$x \approx 18$

a) Virtual = 18 bits

b) Physical = 16 bits

VA : 140×1024

: 143360

2^x : 143360

$\log 2^x = \log 143360$

$x = \frac{\log 143360}{\log 2}$

x : 18 bits

PA : 1024×64

: 65536

2^x : 65536

$\log 2^x = \log 65536$

$x = \frac{\log 65536}{\log 2}$

x : 16 bits

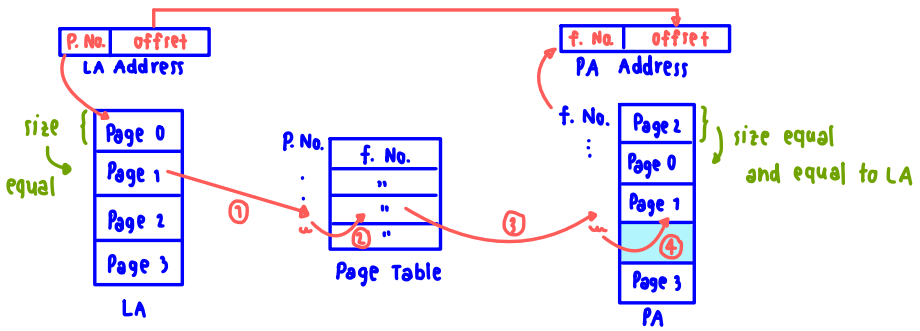


3. การจองพื้นที่ในหน่วยความจำหลัก (PA) โดยใช้แนวคิดที่เรียกว่า Paging Scheme จำเป็นต้องใช้ Page Table จงอธิบายถึงความสำคัญของ Page Table ส่วนการเพิ่มประสิทธิภาพการทำงานของ Page Table สามารถใช้เทคนิคที่เรียกว่า Inverted Page Table จงอธิบายหลักการทำงานของ Inverted Page Table และแง่มุมในการเพิ่มประสิทธิภาพ (5 คะแนน)

Page Table (map ระหว่าง PA & LA) => Use to translate Logical Address to Physical Address for each Process

=> 1 Process = 1 Page Table.

=> The size of each page is equal, because Paging Scheme use to eliminate external fragmentation that occur with Segmentation.



=> Step to access in Physical Address.

1. which page ? from LA that want to execute
2. map to P. No. (Page Number) as index of Page Table.
3. use f. No. (Frame Number) of that page to map to PA.

=> The size of all pages, both in LA, PA are equal. (No external fragmentation)

=> The number of row in page table is equal to the number of page in LA
- Depend on LA ⟶ Large + have 1 Table / 1 process
∴ Spend a lot of space in main memory.

Then, How to reduce the size of page table ?

Solution : Using Invert Page Table.

Invert page table => Reduce the size of page table

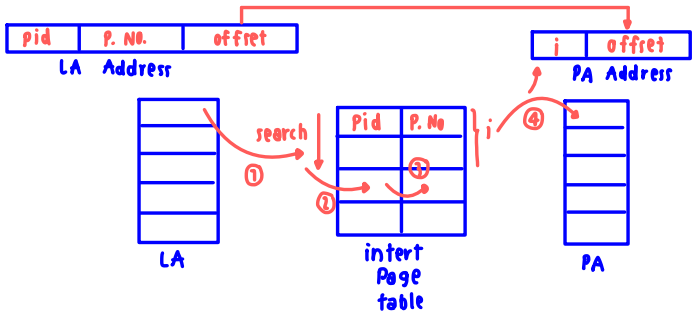
(ใช้การ search pid เมื่อไปหา id แล้วจะได้เลข Index มาเชื่อมกับแผ่น Frame table ช่วยลดพื้นที่และเพิ่มความเร็วในการเข้าถึง)

=> Base on using the number of frames < pages

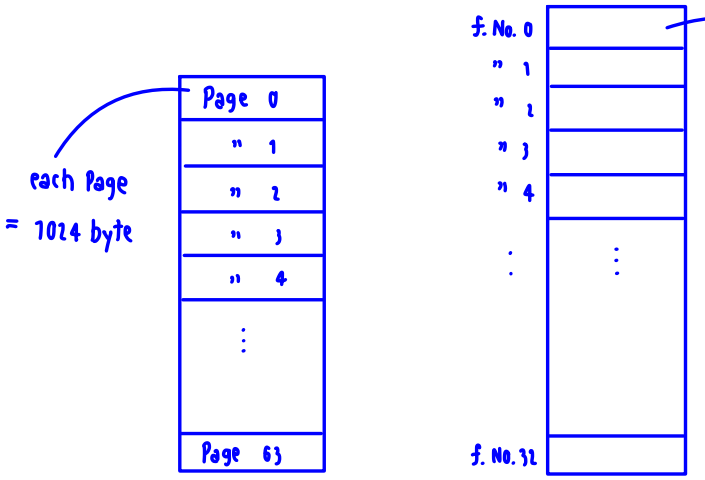
=> 1 entry in invert page table = 1 frame page in PA
∴ The number of row in invert page table = the number of frame page.

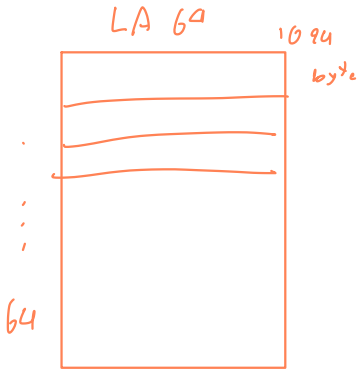
=> each entry consist of P.No. and process identifier (pid) to tell that page is owns of which process.

=> have to search for pid. = increase translation time.



4. พิจารณาระบบปฏิบัติการที่ใช้การจัดการหน่วยความจำหลักแบบ Paging Scheme มี Logical Address 64 เพจ แต่ละเพจมีขนาด 1024 ไบต์ ซึ่งทำงานคู่กับหน่วยความจำหลักที่มี Physical Address 32 เฟรม จงบอกจำนวนบิตของ Logical Address และ Physical Address (2 คะแนน)





LA => 1. find size of LA = 1024 (byte) × 64 (page) = 65,536

2. find How many bit to define LA Address ?

$2^x = 65536$

$\log 2^x = \log 65,536$

$x = \frac{\log 65536}{\log 2}$

$x = 16$ #

3. Separate into 2 part

- bit to define page number => $2^x = 64 \rightarrow x = 6$ #
- bit to define offset => $16 - 6 = 10$ #

LA $64 \times 1024 = 2^x$

$\log_2(1024 \times 64) = x$

$x = 16$

page = LA

frame = PA

จน. bit PA

$32 \times 1024 = 2^x$: $\log_2(1024 \times 32) = x$

$x = 15$

ถ้าหา byte ที่น้อย ขนาด 8

เช่น LA = 16 bit

แปลงเป็น byte : $\frac{16}{8} = 2$ byte

PA => 1. find size of PA = 1024 × 32 = 32768

2 How many bit to define PA Address

$2^x = 32768$

$x = \frac{\log 32768}{\log 2}$

$x = 15$

3. Separate into 2 part

- bit to define frame number => $2^x = 32 \rightarrow x = 5$ #
- bit to define offset => $15 - 5 = 10$ #

Ans => LA = 16 bits, PA = 15 bits. #

6 bits P. No.	10 bits offset

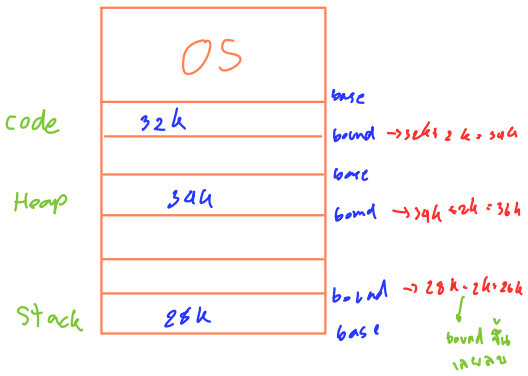
LA Address (16 bits)

5 bits f. No.	10 bits offset

PA Address (15 bits)

5. จงคำนวณหา Physical Address จาก Logical Address ขนาด 16 บิต เมื่อระบบปฏิบัติการใช้วิธี Segmentation เพื่อจองพื้นที่ในหน่วยความจำหลัก และมี Segment Table ดังตารางด้านล่าง โดยสมมุติให้ 4 บิตบนเป็นตัวบอกเซกเมนต์ (ต้องตอบเป็นฐานสิบ) (5 คะแนน)

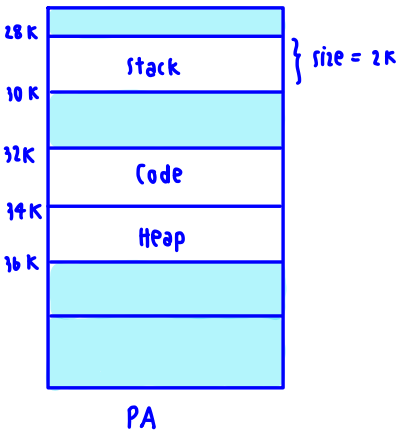
Segment	Base	Size
Code	32K	2K
Heap	34K	2K
Stack	28K	2K



5.1. Logical Address เป็น 320 (เลขฐานสิบ) PA = (32 × 1024) + 320 = 33088

5.2. Logical Address เป็น 2561 (เลขฐานสิบ) PA = (32 × 1024) + 2561 = 35329

5.3. Logical Address เป็น 12528 (เลขฐานสิบ) PA = (32 × 1024) + 12528 = 45296



PA = Base + offset

1. find offset

offset = LA − starting point

1.1) Stack = 320 − 256 = 64 (2^8)

1.2) Code = 2561 − 2048 = 513 (2^{11})

1.3) Heap = 12528 − 8192 = 4336 (2^{13})

2. find PA

PA = Base + offset

2.1) PA = 28K + 64 = 28064

2.2) PA = 32K + 513 = 32513

2.3) PA = 34K + 4336 = 38336

Ans : PA of each

- Stack => 28,064 #
- Code => 32,513 #
- Heap => 38,336 #